Protocolo compartido

SEGUNDA ENTREGA: PROYECTO INTEGRADOR

Jason Montenegro, Jafet Araya, Ivan Ríos

File System

Capacidad de escritura, lectura y borrado de archivos creacion de directorios (mkdir)

División de Super bloque, Bitmap e Inodos y partición de bloque de 256 bytes.

Manejo y encapsulación de errores, y estructura con crecimiento dinámico y cambio de discos.



File System

Super block	Bitmap			Inode table		Data blocks		
file_system_identifier total_blocks block_size inode_table_start inode_count data_start blocks_available inodes_available padding[3]	[-][-][-]...[-]	...	[-][-][-]...[-]	Inode_type inode_id total_size created_timestamp direct_blocks[15] indirect_blocks[15] double_indirect_blocks[15] triple_indirect_blocks[15] padding[3]	...	Raw data	...	Raw data

Simulación

- Cliente: envia soliicitudes, presenta información, utiliza menú con operaciones disponibles.
- Intermediario: interpreta el tipo de solicitud, consulta tabla global, envía y recibe del servidor.
- Servidor: verifica existencia, envia respuesta o error.



Simulación

Utilización de tipos de mensajes, de respuesta y de petición, implementados como enums.

Utilización **de hilos** para funcionamiento, y sincronización por medio mutex, variables de condición.

Bitácoras adaptadas al protocolo compartido: [ENTIDAD] [FECHA_HORA] [TIPO_LOG enrut-resp-sol-err] [MENSAJE]

Tabla Figuras

- Permite responder localmente peticiones de FIGURE_LIST.
- Permite tener un control de lo que está, sin consultar almacenamiento

```
// server -> interm
FIGURE_FOUND,
FIGURE_NOT_FOUND,
// interm -> cliente
FIGURES_LIST,
RETURN_FIGURE,
// Errores
ERROR
```

```
//cliente -> inter
LIST_FIGURES, // s
GET_FIGURE,   // s
// inter -> server
ASK_FIGURE,   //so
//control
STOP
```



Ciclo esperado

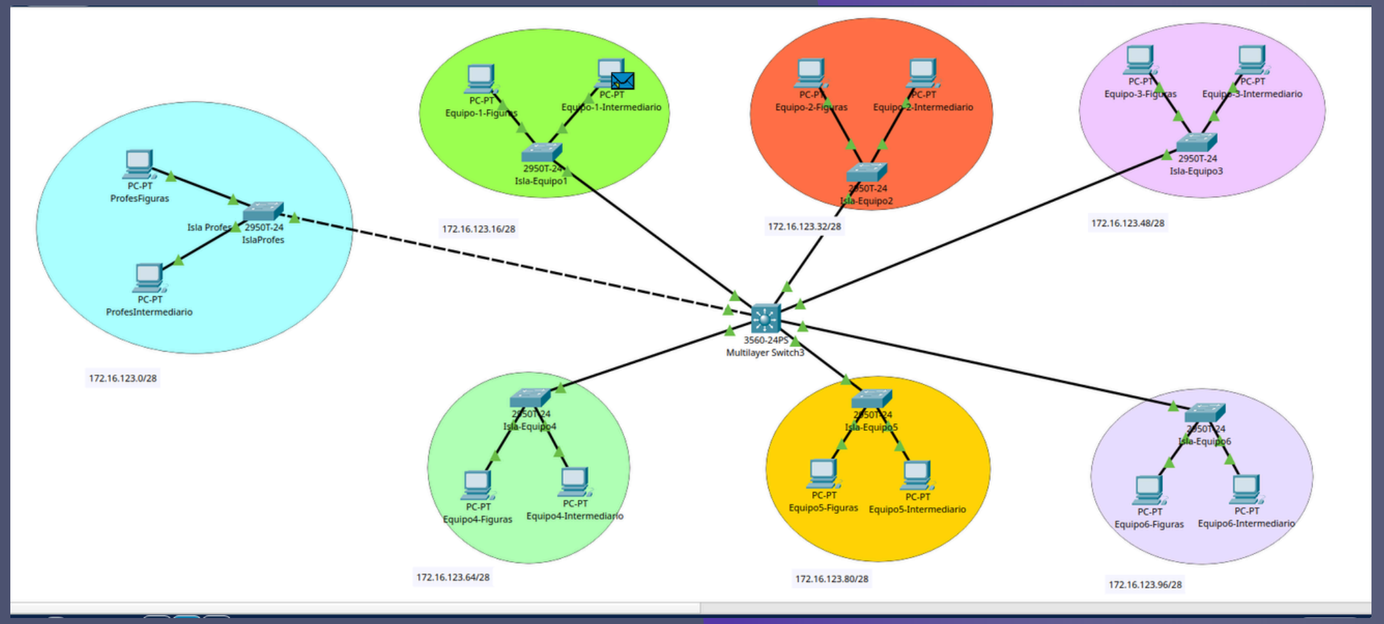
- **Consulta** la tabla de figuras globales.
- Si **figura no se encuentra** en la tabla de figuras, retorna FIGURE_NOT_FOUND.
- De encontrar, la busca en su **propio servidor de figuras**.
- En caso de no estar en el servidor del intermediario, implementa un reenvío del request (fork() - multicast) hacia el resto de intermediarios.
- Si obtiene figura del otro intermediario se vuelve a comunicar en el canal.
- Si no la encuentra eliminar de su tabla de figuras globales y retorna: **FIGURE_NOT_FOUND**

Packet Tracer

Asignacion de VLANs de laboratorio, con puertos troncales.

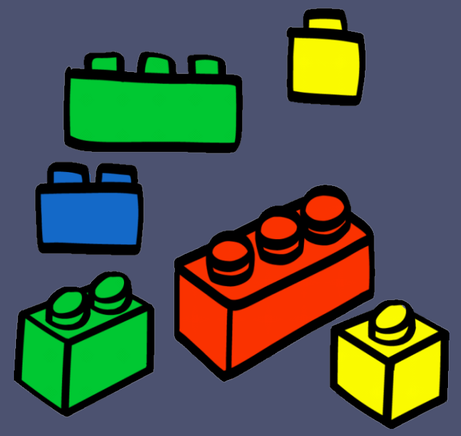
Comunicación de islas con respectivo, cliente, servidor intermediario.

Switch que permite comunicación intermediarios.



¡LEGO SERVER!

SOLICITUDES HTTP/HTTPS



PUERTOS LEGO_SERVER

PUERTO IPv4 con SSL = 8000

PUERTO IPv4 HTTP = 8501

PUERTO IPv6 con SSL= 9002

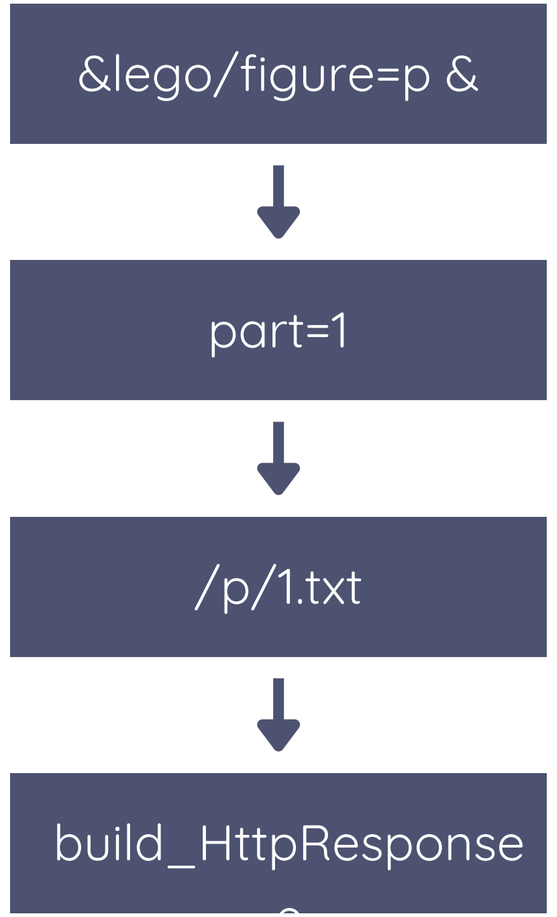
PUERTO IPv6 con HTTP= 9503

SOLICITUDES CLIENTE

Recibe y parsea la solicitud, si es Request verifica que tenga Get al incio, despues busca la solicitud.

extrae del header , la figura y el numero de figura (en caso de que venga list/figures devuelve lista de figuras)

Crea la ruta en el file system con base al nombre de la figura y la parte .



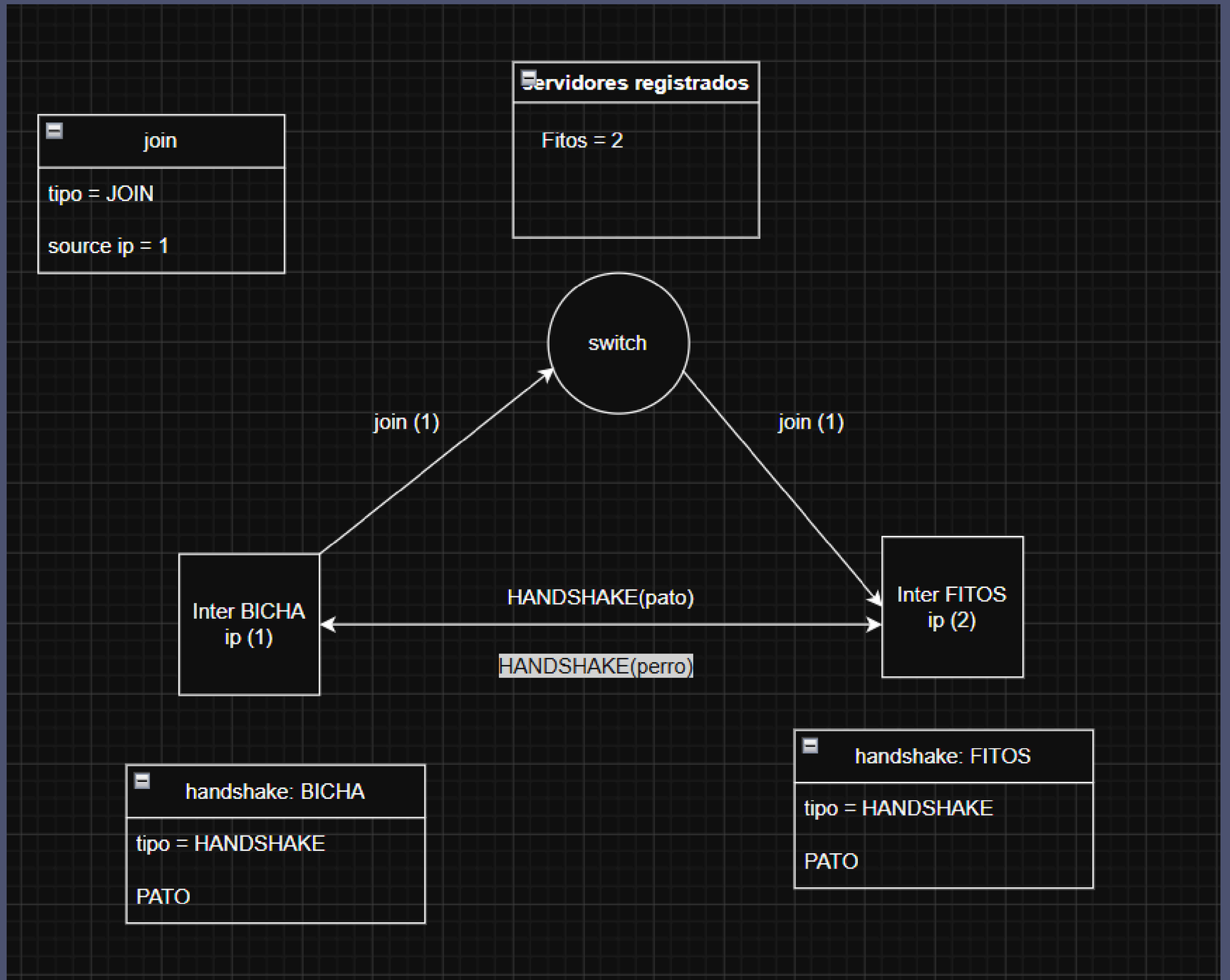
CONSTRUCCIÓN RESPUESTA

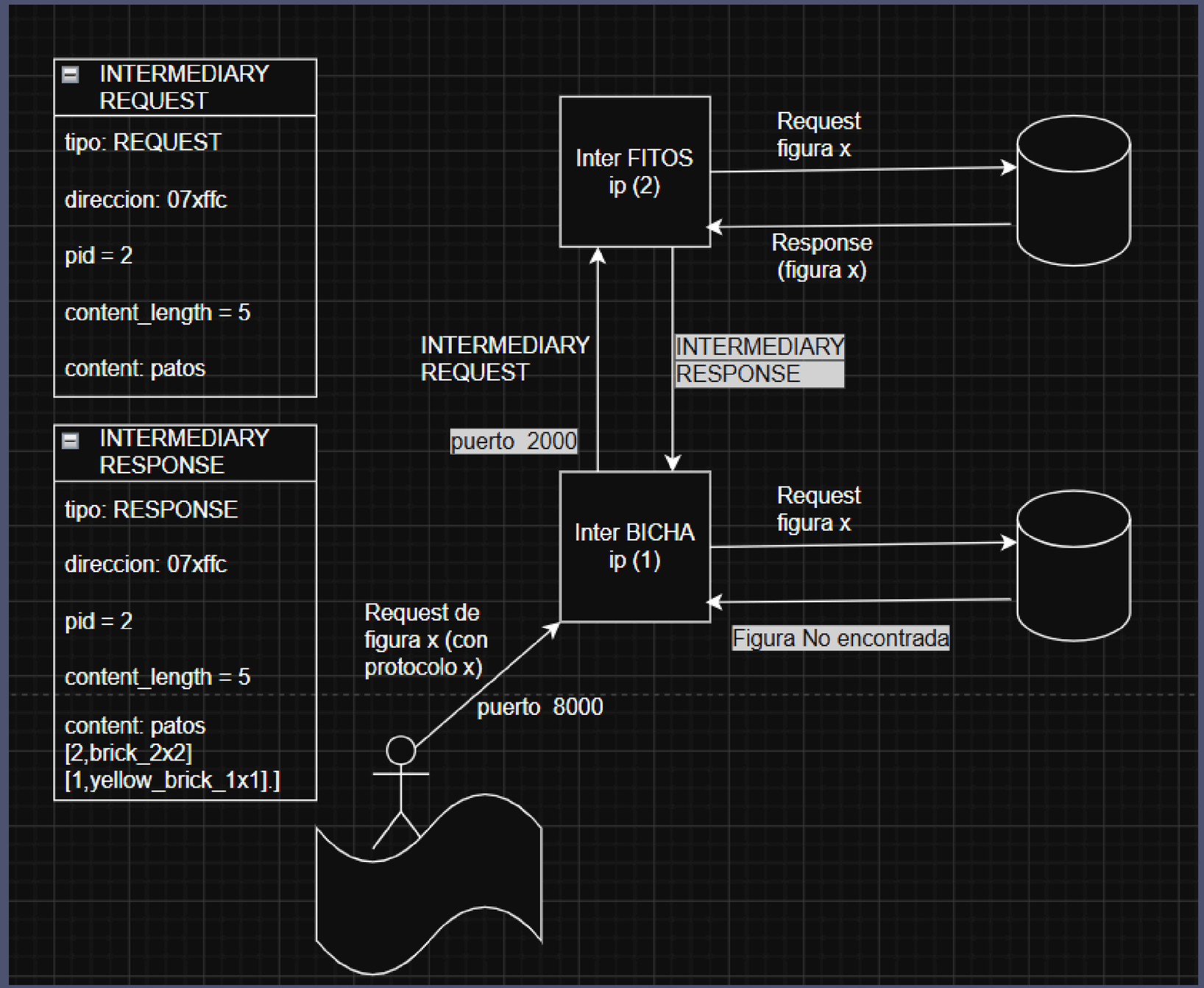
```
std::string Lego_Server::build_HttpResponse(
    const std::string statusCode,  // "200 OK", "404 Not Found", 100 ERROR.
    uint32_t length, std::string contentType, std::string type_conect,
    std::string& content) {
  return "HTTP/1.1 " + statusCode + "\r\n" + "Content-Type: " + contentType +
         "; charset=UTF-8\r\n" + "Content-Length: " + std::to_string(length) +
         "\r\n" + "Connection: " + type_conect + "\r\n" + "\r\n" + content;
}
```

```
build_HttpResponse("404 Not Found", errorBody.size(), "text/html",
                   connectionType, errorBody);
```

```
    build_HttpResponse("200 OK", body_html.size(), "text/html",
                       connectionType, body_html);
//  std::cout << "envio respuesta" << response << std::endl;
client->Write(response.c_str());  // enviadito
```




join
tipo = JOIN
source ip = 1
Servidores registrados
Fitos = 2
switch
join (1)
join (1)
Inter BICHA
ip (1)
HANDSHAKE(pato)
HANDSHAKE(perro)
Inter FITOS
ip (2)
handshake: BICHA
tipo = HANDSHAKE
PATO
handshake: FITOS
tipo = HANDSHAKE
PATO


INTERMEDIARY REQUEST
tipo: REQUEST
direccion: 07xffc
pid = 2
content_length = 5
content: patos
INTERMEDIARY RESPONSE
tipo: RESPONSE
direccion: 07xffc
pid = 2
content_length = 5
content: patos
[2,brick_2x2]
[1,yellow_brick_1x1].]
Inter FITOS
ip (2)
Request
figura x
Response
(figura x)
INTERMEDIARY
REQUEST
INTERMEDIARY
RESPONSE
puerto 2000
Inter BICHA
ip (1)
Request
figura x
Figura No encontrada
Request de
figura x (con
protocolo x)
puerto 8000

¡Presentación de fucionalidades!

